SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: Enterprise Banking System
Authors: K. Pradyun, K. Uday Kumar Goud

1. Introduction

1.1. Purpose:

This SRS document describes the functional and non-functional requirements of the Enterprise Banking System. The purpose of this system is to manage secure financial activities, including money transfers, smart savings, and milestone-based service contracts, protected by an automated real-time fraud detection engine.

1.2. Scope:

- It allows Users to register, complete KYC, view account balances, deposit funds, execute secure transfers, manage Keeper Vaults, and execute Milestone Contracts.
- It allows the System (Fraud Engine) to analyze transactions in real-time, generate risk scores, and approve or freeze funds.
- It allows System Admins to review flagged fraudulent transactions, verify KYC identity documents, and activate user accounts.

1.3. Definitions, Acronyms, Abbreviations:

- SRS – Software Requirement Specifications
- UI – User Interface
- DFD – Data Flow Diagram
- DB – Database
- Admin – System Administrator
- ACID – Atomicity, Consistency, Isolation, Durability
- KYC – Know Your Customer

1.4. References:

Project Feasibility Report, Use Case Diagram of Enterprise Banking System, Data Flow Diagrams (DFD Level 0, Level 1, Level 2), PostgreSQL Official Documentation, Django/React Official Documentation.

1.5. Overall Description:

The system is an online digital banking platform that integrates standard financial transactions with advanced financial discipline tools (Keeper Vaults and Escrow Contracts) under a single system. It manages transaction processing, real-time fraud validation, conditional vault unlocking, and stage-based contract payments.

---

2. Overall Description

2.1. Product Perspective:

Enterprise Banking System is a standalone web-based system that uses a centralized PostgreSQL database, provides role-based access control for different types of users, and operates on a decoupled client-server architecture using ReactJS and Django with asynchronous Celery workers.

2.2. Product Functions:

- User: Authenticate, view balance, deposit funds, transfer funds, create/unlock Keeper Vaults, create/release milestone contracts.
- System Admin: Verify KYC, review flagged transactions, manage accounts.
- System (Fraud Engine): Analyze transaction risk, calculate velocity/location scores, approve or block transfers automatically.

2.3. User Characteristics:

- User: Basic computer and digital financial knowledge, internet access.
- Admin: Technical knowledge, system management skills, fraud verification experience.

2.4. User Constraints:

- Internet connection required.
- Users must have valid login credentials and approved KYC.
- Transfers and contracts strictly require non-negative wallet balances.

2.5. Assumptions and Dependencies:

- Users have valid login credentials.
- System depends on continuous server and Redis broker availability.
- Database connectivity must be active and ACID compliant.
- Third-party gateways for external deposits and SMS/Email notifications are operational.

2.6. Apportioning Requirements:

Future enhancements may include:

- Mobile application version (iOS/Android).
- AI-based spending recommendation system.
- Integration with hardware ATMs.

---

3. Specific Requirements

3.1 Interface Requirements:

3.1.1 External Interface:

- Web browser interface (Chrome, Firefox, Edge, Safari)
- Responsive UI for desktop and mobile devices

3.1.2 Hardware Interface:

- Server system (multi-core for asynchronous processing)
- Client device (PC/Laptop/Tablet/Mobile)
- Internet connectivity

3.1.3 Software Interface:

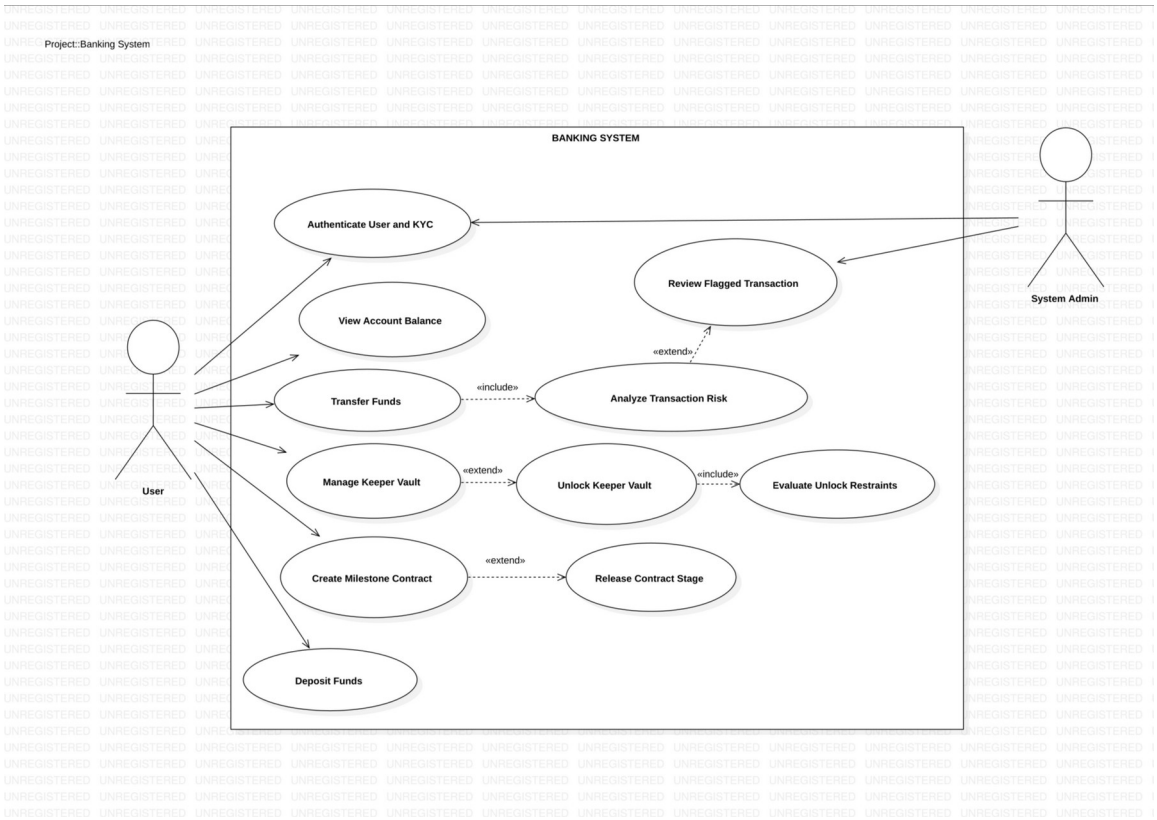
- Web server (Nginx/Apache)
- Backend (Python/Django, Celery workers)
- Message Broker (Redis)
- Database (PostgreSQL)

3.1.4 Communication Interface:

- HTTP/HTTPS protocol
- Secure login authentication (JWT/Sessions)
- SMTP protocol for email notifications and fraud alerts

3.2. Functional Requirements

3.2.1 Use Case Model / Information Flows:



3.2.2 Use Case Specifications:

Case 1

Field	Details
Use Case ID	UC-01
Use Case Name	Authenticate User
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	User
Description	The system verifies the user's credentials to grant secure access to the enterprise banking dashboard.
Preconditions	The user has registered and their KYC is verified by the Administrator.

Field	Details
Post Conditions	The user is securely logged into their session.
Normal Flow	1. User navigates to login page. 2. Enters ID and password. 3. System validates credentials. 4. System grants access.
Alternate Flows	A1: None.
Exceptions	Invalid Credentials: The system denies access. Account temporarily locked after 3 failed attempts.
Priority	High
Frequency	Multiple times per day
Business Rules	Passwords must be hashed. Sessions timeout after 15 minutes.
Assumptions	User has a stable internet connection.

Case 2

Field	Details
Use Case ID	UC-02
Use Case Name	View Account Balance
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	User
Description	The user checks their current main wallet balance and transaction history.
Preconditions	The user is authenticated.
Post Conditions	The current balance is displayed accurately.
Normal Flow	1. User navigates to Dashboard. 2. System retrieves latest balance from DB. 3. System renders data on UI.
Alternate Flows	A1: None.
Exceptions	Database Timeout: Displays "Unable to fetch balance."
Priority	High

Field	Details
Frequency	High
Business Rules	Balances must reflect real-time, ACID-compliant data.
Assumptions	Database read operations are highly available.

Case 3

Field	Details
Use Case ID	UC-03
Use Case Name	Deposit Funds
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	User
Description	The user adds external money to their main wallet.
Preconditions	The user is authenticated.
Post Conditions	Main wallet balance is increased by the deposited amount.
Normal Flow	1. Click "Deposit Funds". 2. Enter amount and payment method. 3. System processes via gateway. 4. Balance updated.
Alternate Flows	A1: None.
Exceptions	Gateway Failure: External payment fails, transaction rolled back.
Priority	Medium
Frequency	Medium
Business Rules	Deposits must be logged immediately for auditing.
Assumptions	Third-party payment gateways are operational.

Case 4

Field	Details
Use Case ID	UC-04

Field	Details
Use Case Name	Transfer Funds
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	User, System (Fraud Engine)
Description	A user initiates a secure transfer, triggering async risk analysis before finalization.
Preconditions	User authenticated, passed KYC, has sufficient balance.
Post Conditions	Funds transferred, balances updated, receipt generated.
Normal Flow	1. Navigate to Transfer. 2. Enter Receiver ID & amount. 3. System verifies balance. 4. Trigger UC-05. 5. Execute atomic transfer.
Alternate Flows	A1: Insufficient Balance. A2: Invalid Receiver ID.
Exceptions	Fraud Flagged: Transfer halted, alerted to Admin.
Priority	High
Frequency	High
Business Rules	Must pass Fraud Engine. Must strictly follow ACID principles.
Assumptions	Celery workers are available.

Case 5

Field	Details
Use Case ID	UC-05
Use Case Name	Analyze Transaction Risk
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	System (Fraud Engine)

Field	Details
Description	Async Celery task calculates risk score based on velocity and location.
Preconditions	Transaction initiated in PENDING state.
Post Conditions	Transaction marked APPROVED or FLAGGED.
Normal Flow	1. Send payload to Redis. 2. Celery analyzes velocity/IP. 3. Risk score calculated. 4. Status updated to APPROVED.
Alternate Flows	A1: None.
Exceptions	High Risk: Score exceeds threshold. Transaction FLAGGED.
Priority	High
Frequency	High
Business Rules	Processing must take under 2 seconds.
Assumptions	Redis and Celery are active.

Case 6

Field	Details
Use Case ID	UC-06
Use Case Name	Manage Keeper Vault
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	User
Description	Creates Smart Savings Vaults with specific goals/locks.
Preconditions	User is authenticated.
Post Conditions	Vault created, funds segregated from main wallet.
Normal Flow	1. Click "Create Vault". 2. Input amount, target date, threshold. 3. Verify balance. 4. Deduct funds, lock Vault.
Alternate Flows	A1: User views existing vaults.

Field	Details
Exceptions	Insufficient Funds.
Priority	Medium
Frequency	Low
Business Rules	Vault funds cannot be used for standard transfers.
Assumptions	Database correctly segregates balances.

Case 7

Field	Details
Use Case ID	UC-07
Use Case Name	Unlock Keeper Vault
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	User
Description	User attempts to withdraw from vault, triggering UC-08.
Preconditions	User has active, locked Vault.
Post Conditions	Vault unlocked, funds returned to main wallet.
Normal Flow	1. Click "Unlock". 2. Trigger UC-08. 3. Evaluation is TRUE. 4. Transfer balance back to main wallet.
Alternate Flows	A1: None.
Exceptions	Restraints Failed: System denies withdrawal.
Priority	Medium
Frequency	Low
Business Rules	Manual overrides are strictly prohibited.
Assumptions	Constraints are correctly set.

Case 8

Field	Details
Use Case ID	UC-08
Use Case Name	Evaluate Unlock Restraints
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	System
Description	Evaluates if vault constraints (Time/Balance) are met.
Preconditions	User requests unlock (UC-07).
Post Conditions	Returns TRUE or FALSE.
Normal Flow	1. Retrieve constraints. 2. Check Date. 3. Check Main Balance. 4. Return TRUE if either is met.
Alternate Flows	A1: If both false, return FALSE.
Exceptions	None.
Priority	High
Frequency	Medium
Business Rules	Queries must hit real-time state of the DB.
Assumptions	Server time is synchronized.

Case 9

Field	Details
Use Case ID	UC-09
Use Case Name	Create Milestone Contract
Created By	K.Pradyun, K.Uday Kumar Goud

Field	Details
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	User
Description	Initiates a stage-based service contract (Escrow/Credit).
Preconditions	User is authenticated.
Post Conditions	Contract created, pending mutual acceptance.
Normal Flow	1. Enter Provider ID, Amount, Stages. 2. Select Escrow. 3. Deduct total to System Escrow. 4. Save Contract.
Alternate Flows	A1: Select Credit mode (Skip deductions).
Exceptions	Insufficient Escrow Funds.
Priority	High
Frequency	Medium
Business Rules	Escrow funds cannot be used by the platform.
Assumptions	Provider exists in the system.

Case 10

Field	Details
Use Case ID	UC-10
Use Case Name	Release Contract Stage
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	User
Description	Releases locked funds to provider for completed stage.
Preconditions	Contract is active and accepted.

Field	Details
Post Conditions	Funds transferred to provider.
Normal Flow	1. Review stages. 2. Click "Release Stage X". 3. Transfer Escrow to Provider. 4. Mark stage COMPLETED.
Alternate Flows	A1: Credit Mode (Deduct directly from main wallet at runtime).
Exceptions	Insufficient Funds during Credit mode release.
Priority	High
Frequency	Medium
Business Rules	Stage releases are irreversible.
Assumptions	Work verification happens outside the system.

Case 11

Field	Details
Use Case ID	UC-11
Use Case Name	Review Flagged Transaction
Created By	K.Pradyun, K.Uday Kumar Goud
Last Updated By	K.Pradyun, K.Uday Kumar Goud
Date Created	26-02-2026
Date Last Updated	26-02-2026
Actors	System Admin
Description	Administrator reviews transaction flagged by Fraud Engine.
Preconditions	Transaction is FROZEN. Admin authenticated.
Post Conditions	Transaction manually approved or permanently blocked.
Normal Flow	1. Navigate to Security Dashboard. 2. Select FROZEN record. 3. Verify user intent. 4. Click Approve. 5. Execute.
Alternate Flows	A1: Click "Block", cancel transaction permanently.
Exceptions	None.
Priority	High

Field	Details
Frequency	Low
Business Rules	Only Admin roles can access this dashboard.
Assumptions	Admins verify via external channels (Phone/Email).

3.2.3 Analysis Classes/Data Dictionary:

Class	Attributes
User	`userID`, `name`, `email`, `kyc_status`, `password_hash`
Account	`accountID`, `userID`, `wallet_balance`, `is_frozen`
Transaction	`transactionID`, `senderID`, `receiverID`, `amount`, `status`, `risk_score`
KeeperVault	`vaultID`, `userID`, `depositAmount`, `targetDate`, `balanceThreshold`, `unlockStatus`
Contract	`contractID`, `creatorID`, `providerID`, `totalAmount`, `fundingMode`, `stageData`

3.3. Performance Requirements

- System should support 1000+ concurrent active sessions.
- Real-time Fraud Engine processing time < 2 seconds.
- Standard API response time < 500 milliseconds.

3.4. Logical Database Requirements

- Relational Database (PostgreSQL).
- Tables: User, Account, Transaction, KeeperVault, MilestoneContract.
- Must enforce strict Foreign Key relationships and constraints (e.g., non-negative numeric constraints on wallet columns).

3.5. Design Constraints

- Decoupled Web-based architecture (React frontend, Django REST API).
- Mandatory use of database transaction blocks (`transaction.atomic()`) for all financial data modifications.
- Role-based authentication (User vs Admin).
- Data privacy regulations and KYC guidelines must be strictly followed.

3.6. Software System Attributes

3.6.1. Reliability

- System uptime ≥ 99.5%.
- In the event of network failure mid-transfer, the atomic transaction must rollback automatically to prevent lost funds.

3.6.2. Availability

- Available 24/7 except during scheduled maintenance.
- Asynchronous architecture ensures the main server UI stays available even if the background risk-engine experiences high traffic.

3.6.3. Security

- Password encryption (PBKDF2/Argon2).
- Role-based access control (RBAC).
- Secure HTTPS communication.
- SQL injection and CSRF prevention.

3.6.4. Maintainability

- Modular Django app architecture.
- Easy to update fraud threshold logic inside Celery workers.
- Scalable relational database design.

3.6.5. Portability

- Compatible with Windows, Linux, and macOS.
- Browser-independent (Chrome, Safari, Firefox).
- Deployable on containerized cloud platforms (Docker/AWS).

---

4. SUPPORTING INFORMATION

The supporting information for the Enterprise Banking System includes the Feasibility Report, Use Case Diagram, Data Flow Diagrams (Level 0, Level 1, Level 2), IEEE SRS standard template, database schema designs, and details of possible future enhancements such as a mobile application and hardware ATM integrations